

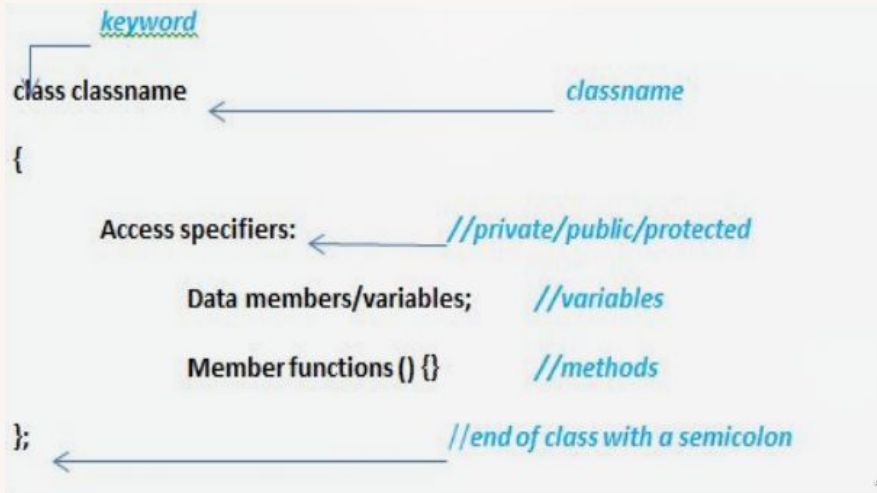
Classes and Enums

Lab 10: https://maryash.github.io/135/labs/lab_10.html

Ryan Vaz

Introduction to Classes

Classes in C++ are user-defined compound datatypes that can be used for grouping together several related variables and functions. These ‘variables’ inside a class are called fields (or data members) of the classes.



```
class classname
{
    Access specifiers:
        Data members/variables;
        Member functions () {}
};
```

The diagram illustrates the syntax of a C++ class with the following annotations:

- keyword**: Points to the `class` keyword.
- classname**: Points to the `classname` identifier.
- Access specifiers:**: Points to the line `//private/public/protected`.
- //variables**: Points to the line `Data members/variables;`.
- //methods**: Points to the line `Member functions () {}`.
- //end of class with a semicolon**: Points to the closing brace and semicolon `};`.

Variables of these class types are referred to as objects. In other words, If you define an instance of your class, you are creating an object of that particular class. Data members and member functions can be private, public or protected depending on the access specifier.

Access Specifiers

public: Can be accessed by anyone who has access to an object of this class. Usually there are getters and setter functions which allows users to modify private fields. More things are usually public including constructors, destructors etc. that are covered in CS235.

private: Can be accessed ONLY by the class itself. Usually sensitive data members and functions are specified as private. Most classes have their data members as private.

protected: Can be accessed by classes that inherit from the current class. This will be covered more in CS235 and maybe later this semester.

Class of Time

```
class Time{
    public: // access specifier
    int h; // hour
    int m; // minute
};

int main(){
    Time now; // empty Time variable
    now.h = 17; // assign hour field
    now.m = 45; // assign minutes field
}
```

This is a class to represent Time. It has two public data members:

- h : hour
- m : minute

The main function creates an empty Time object named now and it updates the h and m fields of this object. The object stores 17:45.

Assignment operator

Instead of creating a new object and updating the fields like the main function in last slide, we can directly create the object with some values. We can also create a new object and make a copy of it from an older object using the assignment operator (=).

```
int main(){
    Time morningLecture = {11, 30};
    Time myAlarm;           // make another variable
    myAlarm = morningLecture; // copying
    cout << morningLecture.h << ":" << morningLecture.m << endl;
    cout << myAlarm.h << ":" << myAlarm.m << endl;
}
```

The default assignment operator works because we have a simple class. In your future courses, you will use more complex classes that will require you to overload the default operators.

Functions with object parameters

You can define functions that take objects as parameters. Here's a useful one:

```
void printTime(Time time){
    cout << time.h << ":" << time.m << endl;
}

int main(){
    Time morningLecture = {11, 30};
    Time myAlarm;
    myAlarm = morningLecture;
    printTime(morningLecture);
    printTime(myAlarm);
}
```

Introducing Enum Types

Now that we are a bit more familiar with classes, we can talk about another similar grouping concept known as Enum

```
enum Genre {ACTION, COMEDY, DRAMA, ROMANCE, THRILLER};
```

Enum types work as sets of named values or labels. A variable of type Genre can assume any of the values listed in the curly braces, example:

```
Genre myFavorite = COMEDY;
```

Just like classes, enums can be used as function parameters, data members and variable types. We will create a class in the next slide using Genre enum as a data member.

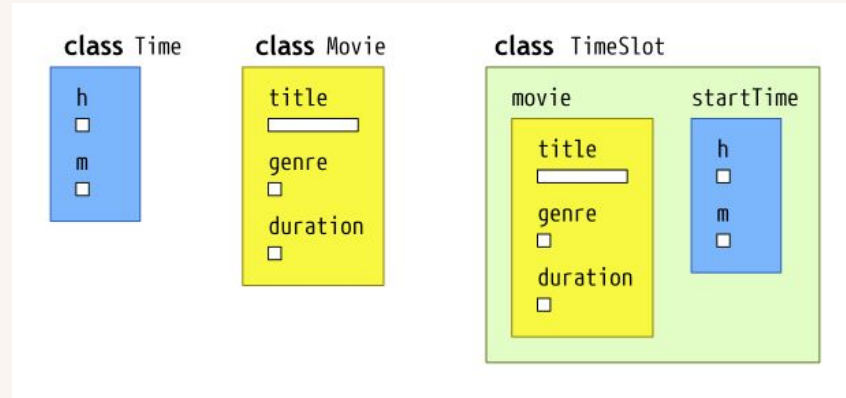
Classes to represent a Movies and Time slots

Using classes and enums as datatypes for class fields:

```
enum Genre {ACTION, COMEDY, DRAMA, ROMANCE, THRILLER};

class Movie{
public:
    string title;
    Genre genre;           // enum of movie
    int duration;          // in minutes
};

class TimeSlot{
public:
    Movie movie;           // what move
    Time startTime;        // when it starts
}
```



Usage example

```
void printMovie(Movie mv){
    string g;
    switch(mv.genre){
        case ACTION:
            g = "Action";
            break;
        case COMEDY:
            g = "Comedy";
            break;
        case DRAMA:
            g = "Drama";
            break;
        case ROMANCE:
            g = "Romance";
            break;
        case THRILLER:
            g = "Thriller";
            break;
    }
    cout << mv.title << " " << g << " (" << mv.duration << " min)";
}
```

```
int main(){
    Movie movie1 = {"Back to the Future", COMEDY, 116};
    Time time1 = {9, 15};
    printMovie(movie1);
    TimeSlot morning = {movie1, time1};
}
```